VERIQTA

LEARN | PRACTICE | INTERVIEW | GROW

DEVOPS • CLOUD • SRE • PLATFORM • INFRASTRUCTURE

REAL SCENARIOS
REAL ANSWERS
REAL JOBS

KUBERNETES INTERVIEW SCENARIOS

50 | FOR MID-LEVEL ENGINEERS

SCENARIOS | EXPLANATIONS | TROUBLESHOOTING | BEST PRACTICES

- **PODS** Troubleshoot workloads
- **DEPLOYMENTS** Manage applications
- **SERVICES** Enable connectivity
- **INGRESS** Expose applications
- **STORAGE** Persistent data
- **SECURITY** RBAC, Secrets
- **NODES** Maintain cluster health
- **MONITORING** Observe and debug
- **SCALING** Ensure availability
- **REAL-WORLD** Production scenarios

INTERVIEW PREP TODAY HIGHER ROLES TOMORROW

- Practical Scenarios
- Step-by-Step Answers
- Real Cluster Examples
- Common Mistakes
- Production Insights
- Interview Tips

THINK | TROUBLESHOOT | SOLVE

LIKE A KUBERNETES ENGINEER

BUILD • DEPLOY • SCALE • RUN

VERIQTA

ENGINEER A BETTER TOMORROW

POD STARTUP FAILURES

REAL SCENARIOS | REAL TROUBLESHOOTING | REAL INTERVIEW QUESTIONS

When a Pod does not start, it is usually due to scheduling, image, configuration, permissions, or application errors. These are common interview scenarios for mid-level Kubernetes engineers.

TROUBLESHOOT LIKE A REAL ENGINEER

1 POD STUCK IN PENDING

SCENARIO

You create a Pod, but it stays in Pending and never starts.

COMMON CAUSES

- Insufficient CPU or memory
- No matching node (nodeSelector, affinity)
- Taints prevent scheduling
- Missing PersistentVolumeClaim
- Image pull secrets not available
- Node not ready

KEY COMMANDS

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl get events --sort-by='.lastTimestamp'
kubectl get nodes
kubectl get pvc
```

WHAT TO LOOK FOR

- Events in describe output
- Scheduling error messages
- Resource requests vs node capacity
- Taints and tolerations
- PVC binding status

EXAMPLE OUTPUT

```
Events:
  Warning  FailedScheduling
  0/3 nodes are available:
  insufficient cpu.
```

HOW TO FIX

- Check resource requests and node capacity
- Add tolerations or adjust nodeSelector
- Fix PVC issues
- Ensure nodes are ready

2 POD ENTERS CRASHLOOPBACKOFF

SCENARIO

A Pod starts but keeps restarting and shows CrashLoopBackOff.

COMMON CAUSES

- Application error or crash
- Wrong configuration or environment variable
- Missing dependency (database, service)
- Health check failure
- Command or arguments are incorrect

KEY COMMANDS

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
```

WHAT TO LOOK FOR

- Exit code in describe output
- Error messages in logs
- Application startup failures
- Incorrect configuration
- Probe failures

EXAMPLE OUTPUT

```
Last State:  Terminated
  Reason:    Error
  Exit Code: 1
  Started:   Mon, 10 Jun 2024 12:00:00
  Finished:  Mon, 10 Jun 2024 12:00:05
```

HOW TO FIX

- Check application logs
- Fix configuration or missing dependencies
- Verify environment variables and secrets
- Test the container locally if possible

3 POD SHOWS IMAGEPULLBACKOFF

SCENARIO

A Pod fails to start and shows ImagePullBackOff.

COMMON CAUSES

- Image name or tag is incorrect
- Image does not exist in the registry
- No access to private registry
- Missing or incorrect imagePullSecret
- Network/DNS issues to the registry

KEY COMMANDS

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl get events
kubectl get secret
```

WHAT TO LOOK FOR

- Image pull error messages in events
- Registry authentication errors
- Incorrect image name or tag
- Missing imagePullSecret

EXAMPLE OUTPUT

```
Events:
  Warning  Failed   Failed to pull image
                    "myapp:latest":
                    rpc error: not found
```

HOW TO FIX

- Verify the image name and tag
- Ensure the image exists in the registry
- Check and create imagePullSecret for private images
- Verify network access to the registry
- Test pulling the image manually

JUNIOR ENGINEER TIP

Always check kubectl describe and events. They usually tell you exactly why a Pod is not starting.

PRODUCTION AWARENESS

In real environments, Pod startup issues can be caused by cluster capacity, image registry access, network policies, or missing dependencies. Always check the events and logs before making changes.

COMMON MISTAKE

Focusing only on the Pod status and not checking events, logs, or node conditions.

CONTAINER RUNTIME PROBLEMS

REAL SCENARIOS | REAL TROUBLESHOOTING | REAL INTERVIEW QUESTIONS

Container runtime problems occur when the container process fails to start, crashes, or behaves differently in the cluster compared to your local environment. These scenarios test your ability to investigate logs, configuration, environment, and cluster behavior.

TROUBLESHOOT LIKE A REAL ENGINEER

4 CONTAINER EXITS IMMEDIATELY AFTER STARTUP

SCENARIO

A Pod starts but the container exits immediately with a non-zero exit code.

COMMON CAUSES

- Application error or misconfiguration
- Missing environment variables
- Command or arguments are incorrect
- Required files or dependencies missing
- Script exits after completing a task

KEY COMMANDS

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
```

WHAT TO LOOK FOR

- Exit code in describe output
- Error messages in logs
- Missing configuration or secrets
- Incorrect command or arguments
- Application startup failures

EXAMPLE OUTPUT

```
Last State:   Terminated
Reason:       Error
Exit Code:    1
Started:      Mon, 10 Jun 2024 10:00:00
Finished:     Mon, 10 Jun 2024 10:00:02
```

HOW TO FIX

- Check container logs for the real error
- Verify environment variables and secrets
- Fix the command or arguments
- Ensure required files and dependencies exist
- Update the image if needed

5 CONTAINER IS REPEATEDLY RESTARTED

SCENARIO

The Pod keeps restarting and shows increasing restart count.

COMMON CAUSES

- Application crash
- Uncaught exception
- Liveness probe failure
- Out of memory (OOMKilled)
- Dependency not ready (database, API)

KEY COMMANDS

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name> --previous
kubectl get events
kubectl top pod <pod-name>
```

WHAT TO LOOK FOR

- Restart count increasing
- Reason for restart (CrashLoopBackOff)
- Error messages in logs
- Probe failures
- Resource issues (CPU/Memory)

EXAMPLE OUTPUT

```
STATUS              RESTARTS      AGE
CrashLoopBackOff    5 (2m ago)    15m
```

HOW TO FIX

- Check logs and previous logs
- Fix application errors
- Adjust probe configuration if needed
- Check resource limits and requests
- Ensure dependencies are available
- Verify the container image and config

6 APPLICATION WORKS LOCALLY BUT FAILS INSIDE THE POD

SCENARIO

The application runs correctly on your local machine but fails inside the Kubernetes Pod.

COMMON CAUSES

- Missing environment variables
- Different configuration or default values
- Missing dependencies or system packages
- File path or permission issues
- Network or DNS differences
- Resource limits or architecture differences

KEY COMMANDS

```
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- /bin/sh
kubectl describe pod <pod-name>
kubectl get configmap,secret
kubectl get events
```

WHAT TO LOOK FOR

- Error messages in logs
- Missing environment variables
- Configuration differences
- File paths and permissions
- Network/DNS connectivity
- Dependencies not present in the image

HOW TO FIX

- Compare environment variables
- Check ConfigMaps and Secrets
- Ensure all dependencies are in the image
- Verify file paths and permissions
- Test network and DNS from inside the Pod
- Use the same base image and architecture
- Reproduce the issue using kubectl exec

JUNIOR ENGINEER TIP

If it works locally but not in the cluster, always check logs, environment variables, configuration, and network access.

SCHEDULING FAILURES

REAL SCENARIOS | REAL TROUBLESHOOTING | REAL INTERVIEW QUESTIONS

Scheduling failures happen when Kubernetes cannot find a suitable node for your Pod based on resources, taints, affinity rules, or other constraints.

TROUBLESHOOT LIKE A REAL ENGINEER

7 POD CANNOT BE SCHEDULED BECAUSE OF INSUFFICIENT CPU

SCENARIO

You create a Pod but it remains in Pending. The cluster does not have enough available CPU to schedule it.

COMMON CAUSES

- Not enough CPU resources on nodes
- Requests are higher than available capacity
- Other Pods are using most of the CPU
- Cluster autoscaler is not enabled
- Incorrect resource requests/limits

KEY COMMANDS

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl get nodes
kubectl top nodes
kubectl get events --sort-by=.lastTimestamp
```

WHAT TO LOOK FOR

- FailedScheduling events
- Insufficient cpu message
- Node allocatable vs requested resources
- Current node CPU usage
- Whether cluster autoscaler is running

EXAMPLE OUTPUT

```
Events:
  Type     Reason            Age   Message
  ----     ------            ---   -------
  Warning  FailedScheduling  2m
  0/3 nodes are available:
  insufficient cpu.
```

HOW TO FIX

- Reduce CPU requests if possible
- Add more nodes or enable cluster autoscaling
- Move non-critical workloads
- Use right-sized instance types
- Verify resource requests and limits

8 TAINTS PREVENT POD SCHEDULING

SCENARIO

You create a Pod but it remains in Pending because the target nodes have a taint that the Pod does not tolerate.

COMMON CAUSES

- Node has a taint (e.g. node-role, NoSchedule)
- Pod does not have a matching toleration
- Taints added for system or dedicated workloads
- Wrong toleration key, value, or effect
- All available nodes are tainted

KEY COMMANDS

```
kubectl describe pod <pod-name>
kubectl get nodes --show-labels
kubectl describe node <node-name>
kubectl get taints
kubectl get events
```

WHAT TO LOOK FOR

- FailedScheduling events
- Taint messages in the events
- Node taints and effects (NoSchedule, PreferNoSchedule)
- Whether the Pod has correct tolerations

EXAMPLE OUTPUT

```
0/3 nodes are available:
3 node(s) had taint
node-role.kubernetes.io/master:NoSchedule
that the pod didn't tolerate.
```

HOW TO FIX

- Add the correct toleration to the Pod
- Remove the taint (if appropriate)
- Use a different node pool
- Verify toleration key, value, and effect
- Ensure workloads are scheduled only on intended nodes

9 NODE AFFINITY LEAVES A WORKLOAD UNSCHEDULABLE

SCENARIO

You create a Pod with node affinity rules, but it remains in Pending because no node matches the specified labels or requirements.

COMMON CAUSES

- No nodes match the required labels
- Using requiredDuringScheduling and no match
- Label keys or values are incorrect
- Node labels are missing
- Too restrictive affinity rules

KEY COMMANDS

```
kubectl describe pod <pod-name>
kubectl get nodes --show-labels
kubectl get events
kubectl get node <node-name> --show-labels
kubectl get pod -o wide
```

WHAT TO LOOK FOR

- FailedScheduling events
- Node affinity mismatch messages
- Required node affinity rules
- Existing node labels
- Whether rules are too strict

EXAMPLE OUTPUT

```
0/3 nodes are available:
3 node(s) didn't match Pod's
node affinity/selector.
```

HOW TO FIX

- Verify node labels and values
- Correct the affinity expression
- Use preferredDuringScheduling if strict matching is not required
- Label the nodes correctly
- Adjust the affinity rules to match available nodes

CPU AND MEMORY PROBLEMS

REAL SCENARIOS | REAL TROUBLESHOOTING | REAL INTERVIEW QUESTIONS

CPU and memory problems can cause poor performance, unexpected restarts, and application failures in Kubernetes. These scenarios test your ability to investigate resource usage, limits, and cluster behavior.

TROUBLESHOOT LIKE A REAL ENGINEER

10 CONTAINER IS OOMKILLED

SCENARIO

A Pod keeps restarting and the container status shows OOMKilled.

COMMON CAUSES

- Memory limit is too low
- Application memory leak
- High memory usage spikes
- Large in-memory data processing
- JVM/Node.js/other runtime using more memory than expected

KEY COMMANDS

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name> --previous
kubectl top pod <pod-name>
kubectl top node
```

WHAT TO LOOK FOR

- Reason: OOMKilled in describe output
- Memory usage vs limit
- Application logs before termination
- Memory usage patterns over time
- Node memory pressure events

EXAMPLE OUTPUT

```
Last State:  Terminated
 Reason:     OOMKilled
 Exit Code:  137
 Started:    Mon, 10 Jun 2024 10:00:00
 Finished:   Mon, 10 Jun 2024 10:00:05
```

HOW TO FIX

- Increase memory limit (after analysis)
- Optimize application memory usage
- Fix memory leaks
- Use appropriate requests and limits
- Monitor memory usage with Prometheus

11 POD IS CPU THROTTLED

SCENARIO

The application is running but performance is slow. Metrics show the container is CPU throttled.

COMMON CAUSES

- CPU limit is set too low
- High CPU usage and throttling by cgroups
- Bursty workloads exceeding CPU limit
- Insufficient node capacity
- No CPU requests set, causing contention

KEY COMMANDS

```
kubectl top pod <pod-name>
kubectl describe pod <pod-name>
kubectl get events
kubectl exec <pod-name> -- cat
        /sys/fs/cgroup/cpu/cpu.stat
```

WHAT TO LOOK FOR

- High CPU usage
- Throttling metrics (cpu.stat)
- Low CPU limits
- Node CPU pressure events
- Application performance issues (latency)

EXAMPLE OUTPUT

```
cpu.stat:
  nr_periods       10000
  nr_throttled     5234
  throttled_time   1256789012
```

HOW TO FIX

- Increase CPU limit or optimize CPU usage
- Set appropriate CPU requests and limits
- Use horizontal scaling for high load
- Check for inefficient code or high CPU processes
- Monitor with Prometheus and Grafana

12 INCORRECT REQUESTS AND LIMITS AFFECT WORKLOAD STABILITY

SCENARIO

The application is unstable. Pods are sometimes evicted or perform poorly due to incorrect resource requests and limits.

COMMON CAUSES

- Requests set too low
- Limits set too low or too high
- No requests or limits defined
- Resource contention on the node
- Not understanding application needs

KEY COMMANDS

```
kubectl describe pod <pod-name>
kubectl get pods -o wide
kubectl top pods
kubectl get nodes
kubectl describe node <node-name>
```

WHAT TO LOOK FOR

- Requests and limits in pod spec
- Node resource allocations
- Eviction events
- Pod QoS class (BestEffort, Burstable, Guaranteed)
- Resource usage vs configured values

EXAMPLE OUTPUT

```
Resources:
  Requests:  cpu: 100m    memory: 128Mi
  Limits:    cpu: 100m    memory: 128Mi
  QoS Class: Burstable
```

HOW TO FIX

- Set realistic requests based on baseline usage
- Set limits to prevent noisy neighbor issues
- Use Vertical Pod Autoscaler (VPA) for recommendations
- Monitor resource usage over time
- Test changes in a non-production environment

JUNIOR ENGINEER TIP
Always set both requests and limits. Start with real metrics, not guesses.

PRODUCTION AWARENESS
Incorrect resource configuration can cause poor performance, pod evictions, and higher infrastructure costs.

REMEMBER
Right-size your resources. Monitor, measure, and adjust as your application and traffic grow.

DEPLOYMENT FAILURES

REAL SCENARIOS | REAL TROUBLESHOOTING | REAL INTERVIEW QUESTIONS

Deployment failures can lead to application downtime, failed releases, and unstable environments. These scenarios test your ability to diagnose rollout issues, understand Kubernetes deployment strategy, and safely recover from failures.

TROUBLESHOOT LIKE A REAL ENGINEER

13 DEPLOYMENT ROLLOUT BECOMES STUCK

SCENARIO

You deploy a new version, but the rollout stays in progress and never completes.

COMMON CAUSES

- Readiness probe failing
- Application not starting correctly
- Insufficient resources on nodes
- Image pull errors
- Misconfiguration (env vars, config, secrets)
- New Pods failing health checks

KEY COMMANDS

```
kubectl get deployment <name>
kubectl describe deployment <name>
kubectl get pods
kubectl describe pod <pod-name>
kubectl get events --sort-by=.lastTimestamp
```

WHAT TO LOOK FOR

- Rollout status and conditions
- Events and error messages
- Pod health (readiness/liveness)
- Image pull errors
- Resource and scheduling issues
- Application logs

EXAMPLE OUTPUT

```
kubectl rollout status deployment/api
Waiting for deployment "api" rollout
to finish: 1 of 3 updated replicas
are available...
```

HOW TO FIX

- Check pod logs and events
- Fix application or configuration issues
- Verify probes and resource requests
- Ensure image is available
- Resolve scheduling issues
- Rerun the rollout

14 NEW RELEASE CREATES UNAVAILABLE REPLICAS

SCENARIO

You deploy a new version, but some or all replicas are not becoming available.

COMMON CAUSES

- Application crash or startup failure
- Readiness probe failing
- Incorrect environment variables
- Missing or wrong ConfigMaps/Secrets
- Database or external dependency failures
- Insufficient resources or node constraints

KEY COMMANDS

```
kubectl get deployment <name>
kubectl get pods -l app=<name>
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get events
```

WHAT TO LOOK FOR

- Pod status (CrashLoopBackOff, Error)
- Readiness probe failures
- Application error logs
- Missing configuration or secrets
- Dependency connection errors
- Events showing the exact failure reason

EXAMPLE OUTPUT

```
NAME                  READY   STATUS
api-7d9f6b7c9-abc     0/1     CrashLoopBackOff
api-7d9f6b7c9-def     0/1     Running
```

HOW TO FIX

- Check application logs
- Fix configuration, env vars, or secrets
- Verify external dependencies (DB, APIs)
- Adjust probes if needed
- Ensure enough cluster resources
- Redeploy and verify the rollout

15 APPLICATION MUST BE SAFELY ROLLED BACK

SCENARIO

A new deployment introduces issues and you need to roll back to the previous stable version.

COMMON CAUSES

- New release has bugs
- Breaking configuration changes
- New image version is unstable
- Database or dependency issues
- Unexpected behavior in production
- Insufficient testing before deployment

KEY COMMANDS

```
kubectl rollout history deployment <name>
kubectl rollout undo deployment <name>
kubectl rollout status deployment <name>
kubectl get pods
kubectl describe deployment <name>
```

WHAT TO LOOK FOR

- Previous revision history
- Confirm the correct version to roll back
- Rollout status after rollback
- Pod health and application logs
- Verify application functionality
- Identify what caused the failure

EXAMPLE OUTPUT

```
kubectl rollout history deployment/api
REVISION    CHANGE-CAUSE
1           Initial deployment
2           image: api:v2
kubectl rollout undo deployment/api
deployment.apps/api rolled back
```

HOW TO FIX

- Identify the last stable revision
- Run rollout undo to the previous version
- Verify pods are healthy
- Check application functionality
- Investigate the root cause before redeploying

JUNIOR ENGINEER TIP
Always keep a rollback plan. Use meaningful image tags and test in a non-production environment.

PRODUCTION AWARENESS
Rolling back fixes the immediate issue, but you must still investigate and fix the root cause.

REMEMBER
A successful deployment is not just about running pods, it's about a healthy, stable and observable application.

SERVICES AND CONNECTIVITY

REAL SCENARIOS | REAL TROUBLESHOOTING | REAL INTERVIEW QUESTIONS

Kubernetes Services provide stable networking for Pods. Connectivity issues are common and can be caused by selectors, endpoints, ports, DNS, network policies, or application problems. These scenarios test your ability to diagnose and fix real service connectivity issues.

TROUBLESHOOT LIKE A REAL ENGINEER

16 SERVICE EXISTS BUT APPLICATION CANNOT BE REACHED

SCENARIO

You create a Service and it exists, but the application is not reachable from inside or outside the cluster.

COMMON CAUSES

- Incorrect Service type (ClusterIP, NodePort, LoadBalancer)
- Wrong port or targetPort
- NetworkPolicy blocking traffic
- Pod not listening on the expected port
- DNS or ingress misconfiguration

KEY COMMANDS

```
kubectl get svc
kubectl describe svc <svc-name>
kubectl get endpoints <svc-name>
kubectl get pods -l <label>
kubectllogs <pod-name>
kubectl exec -it <pod-name> -- curl localhost:<port>
```

WHAT TO LOOK FOR

- Service type and port configuration
- Endpoints exist and match running Pods
- Events and error messages
- NetworkPolicy rules
- Application listening on the correct port

EXAMPLE OUTPUT

```
NAME    TYPE        CLUSTER-IP     PORT(S)
myapp   ClusterIP   10.96.123.45   80/TCP
```

HOW TO FIX

- Verify Service type and ports
- Check that endpoints exist
- Ensure Pods are healthy and listening
- Review NetworkPolicy rules
- Test connectivity from a Pod using curl or wget

17 SERVICE HAS NO ENDPOINTS

SCENARIO

You create a Service, but it shows no endpoints and traffic cannot reach the application.

COMMON CAUSES

- Selector labels do not match any Pods
- Pods are not ready
- Pod labels are missing or incorrect
- Readiness probe is failing
- All Pods are in a different namespace

KEY COMMANDS

```
kubectl get svc <svc-name>
kubectl get endpoints <svc-name>
kubectl describe svc <svc-name>
kubectl get pods -l <label>
kubectl describe pod <pod-name>
```

WHAT TO LOOK FOR

- Endpoints list is empty
- Selector labels vs Pod labels
- Pod readiness status
- Readiness probe failures
- Pods running in the correct namespace

EXAMPLE OUTPUT

```
NAME    ENDPOINTS   AGE
myapp   <none>      10m
```

HOW TO FIX

- Correct the Service selector labels
- Ensure Pods are running and ready
- Fix readiness probe issues
- Verify namespace and labels
- Redeploy the application if needed

18 WRONG TARGETPORT BREAKS TRAFFIC

SCENARIO

The Service exists and has endpoints, but the application is not reachable because the targetPort is incorrect.

COMMON CAUSES

- targetPort does not match container port
- Named port is incorrect or missing
- Application listens on a different port
- Multiple containers with different ports
- Misunderstanding of port vs targetPort

KEY COMMANDS

```
kubectl get svc <svc-name> -o yaml
kubectl get pods <pod-name> -o yaml
kubectl describe pod <pod-name>
kubectl exec -it <pod-name> -- ss -tlnp
kubectl logs <pod-name>
```

WHAT TO LOOK FOR

- Service port and targetPort values
- Container ports in the Pod spec
- Application listening port (ss or netstat)
- Named port matching
- Endpoints showing correct IP and port

EXAMPLE OUTPUT

```
PORT(S)     TARGETPORT
80/TCP      8080
```

HOW TO FIX

- Set the correct targetPort
- Use the correct named port if applicable
- Verify the container actually listens on that port
- Update and apply the Service
- Test connectivity again

JUNIOR ENGINEER TIP
A Service only routes traffic to Pods that are Ready and match the selector. Always check endpoints first.

COMMON MISTAKE
Confusing port and targetPort. port is what the Service exposes, targetPort is the port on the Pod.

PRODUCTION AWARENESS
Service connectivity issues can be caused by network policies, DNS, load balancers, or application health. Always test from inside the cluster first.

KUBERNETES DNS

REAL SCENARIOS | REAL TROUBLESHOOTING | REAL INTERVIEW QUESTIONS

Kubernetes uses CoreDNS for internal name resolution. DNS issues can prevent Pods from reaching Services, external resources, or the Kubernetes API. These scenarios test your ability to diagnose and fix real DNS problems in the cluster.

TROUBLESHOOT LIKE A REAL ENGINEER

19 POD CANNOT RESOLVE A SERVICE NAME

SCENARIO

A Pod cannot resolve a Service name. The application shows "name or service not known" when trying to reach another Service.

COMMON CAUSES

- DNS policy is incorrect
- CoreDNS is not reachable
- Service name is wrong
- Wrong namespace
- NetworkPolicy blocking DNS (port 53)
- CoreDNS configuration issue

KEY COMMANDS

```
kubectl exec -it <pod> -- nslookup <service-name>
kubectl exec -it <pod> -- dig <service-name>
kubectl exec -it <pod> -- cat /etc/resolv.conf
kubectl get svc -n <namespace>
kubectl get endpoints <service-name>
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

WHAT TO LOOK FOR

- DNS resolution errors (NXDOMAIN, timeout)
- Correct service name and namespace
- /etc/resolv.conf contents
- CoreDNS Pod status and logs
- NetworkPolicy rules
- Whether other Pods can resolve the same name

EXAMPLE OUTPUT

```
$ nslookup myapp
Server:    10.96.0.10
Address:   10.96.0.10#53

** server can't find myapp.default.svc.cluster.local:
NXDOMAIN
```

HOW TO FIX

- Use the correct service name and namespace
- Verify CoreDNS is running and healthy
- Check DNS policy (ClusterFirst)
- Ensure NetworkPolicies allow port 53 (UDP/TCP)
- Test resolution using a known working Pod
- Recreate CoreDNS if necessary

20 COREDNS PROBLEMS AFFECT WORKLOADS

SCENARIO

CoreDNS Pods are crashing, not responding, or returning errors. Multiple Pods in the cluster cannot resolve names.

COMMON CAUSES

- CoreDNS Pods are not running or keep restarting
- High memory or CPU usage
- Misconfiguration in CoreDNS ConfigMap
- Upstream DNS issues (external DNS)
- Cluster network problems
- Node issues affecting CoreDNS
- Too many DNS queries (rate limiting)

KEY COMMANDS

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system <coredns-pod>
kubectl describe pod -n kube-system <coredns-pod>
kubectl get configmap -n kube-system coredns -o yaml
kubectl top pods -n kube-system
kubectl get events -n kube-system
kubectl exec -it <pod> -- nslookup kubernetes.default
```

WHAT TO LOOK FOR

- CoreDNS Pod status (Running, CrashLoopBackOff)
- Error messages in logs
- High resource usage
- Misconfigurations in CoreDNS ConfigMap
- Connectivity to upstream DNS servers
- DNS timeouts or SERVFAIL errors
- Cluster events related to CoreDNS

EXAMPLE OUTPUT

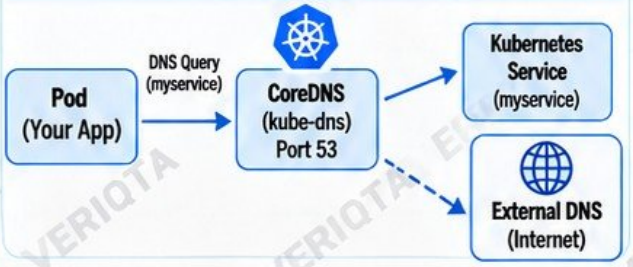
```
$ kubectl get pods  -n kube-system
NAME      READY    STATUS    RESTARTS    AGE
coredns- 6f6b679f8-abcde   0/1
CrashLoopBackOff   5   10m

$ kubectl logs coredns-6f6b679f8-abcde
[ERROR] plugin/errors: 2 myapp.svc.cluster.local.
SERVFAIL
```

HOW TO FIX

- Check CoreDNS logs and identify the root cause
- Verify and fix the CoreDNS ConfigMap
- Ensure sufficient CPU and memory resources
- Check upstream DNS servers are reachable
- Fix network or node issues
- Restart or redeploy CoreDNS if needed
- Monitor CoreDNS after the fix

KUBERNETES DNS ARCHITECTURE



JUNIOR ENGINEER TIP

Always test DNS from inside a Pod using nslookup or dig. If DNS fails, your application will fail even if the Service and Pods are running.

COMMON MISTAKE

Using the wrong service name or namespace. Remember: <Service>.<Namespace>.svc.cluster.local

PRODUCTION AWARENESS

Monitor CoreDNS health, resource usage, and DNS query latency. DNS is a critical dependency for every workload in the cluster.

INGRESS AND EXTERNAL TRAFFIC

REAL SCENARIOS | REAL TROUBLESHOOTING | REAL INTERVIEW QUESTIONS

TROUBLESHOOT LIKE A REAL ENGINEER

Kubernetes Ingress exposes your Services to external traffic, usually through a controller like NGINX Ingress, AWS ALB Ingress, or other ingress controllers. Misconfigurations can cause 404, 502, 503 errors or TLS/HTTPS issues. These scenarios test your ability to troubleshoot real-world ingress and external access problems.

21 INGRESS RETURNS 404

SCENARIO

You have an Ingress configured, but accessing the URL returns 404 Not Found.

COMMON CAUSES

- Incorrect host or path in Ingress rules
- Path mismatch (prefix vs exact)
- Service or backend not found
- Ingress controller misconfiguration
- Wrong ingress class
- Application not running or wrong port

KEY COMMANDS

```
kubectl get ingress
kubectl describe ingress <name>
kubectl get svc
kubectl get endpoints
kubectl get pods
kubectl logs -n <namespace> <pod-name>
```

WHAT TO LOOK FOR

- Ingress rules and paths
- Correct host and service name
- Endpoints exist and match the Service
- Ingress controller logs
- Application is running and listening
- Path rewriting issues

EXAMPLE OUTPUT

```
HOSTS       ADDRESS    PORTS     AGE
myapp.com   1.2.3.4    80,443    10m

HTTP/1.1 404 Not Found
Server: nginx
```

HOW TO FIX

- Verify host and path in Ingress manifest
- Ensure Service and endpoints exist
- Check ingress controller configuration
- Validate correct ingress class is used
- Confirm application is running and healthy
- Test using curl with the correct host header

22 INGRESS RETURNS 502 OR 503

SCENARIO

You access the application through the Ingress, but it returns 502 Bad Gateway or 503 Service Unavailable.

COMMON CAUSES

- Pods are not ready or unhealthy
- Service endpoints are empty
- Ingress controller cannot reach backend
- Application crashes or slow to respond
- Resource limits or high load
- Network policy or security group blocking

KEY COMMANDS

```
kubectl get pods
kubectl get endpoints <service-name>
kubectl describe svc <service-name>
kubectl logs -n <namespace> <pod-name>
kubectl logs -n <namespace> -l app=nginx
kubectl get events
```

WHAT TO LOOK FOR

- Pod readiness and health checks
- Endpoints and target ports
- Ingress controller error logs
- Upstream connection failures
- High latency or timeouts
- Resource usage (CPU, memory)

EXAMPLE OUTPUT

```
HTTP/1.1 502 Bad Gateway
Server: nginx

HTTP/1.1 503 Service Unavailable
upstream temporarily unavailable
```

HOW TO FIX

- Ensure Pods are healthy and ready
- Check Service endpoints
- Verify Service port and targetPort
- Review ingress controller logs
- Increase resources if needed
- Check network policies and firewall rules
- Fix application errors or startup delays

23 TLS CERTIFICATE OR HTTPS CONFIGURATION FAILS

SCENARIO

You configure HTTPS for your Ingress, but you get certificate errors, a browser warning, or TLS handshake failures.

COMMON CAUSES

- Invalid or expired certificate
- Incorrect TLS secret or secret name
- Hostname does not match certificate (CN/SAN)
- Certificate not issued or not yet ready (cert-manager)
- Ingress controller not configured for TLS
- Mixed content or redirect issues

KEY COMMANDS

```
kubectl get secret
kubectl describe secret <tls-secret>
kubectl describe ingress <name>
kubectl logs -n <namespace> -l app=ingress-nginx
kubectl get certificate
kubectl describe certificate <name>
```

WHAT TO LOOK FOR

- Certificate status and expiration date
- Correct secret name referenced in Ingress
- Hostname matches certificate (CN/SAN)
- Certificate events (cert-manager)
- Ingress controller TLS configuration
- Redirect and SSL termination settings

EXAMPLE OUTPUT

```
$ openssl s_client -connect myapp.com:443
verify error:num=10:certificate has expired
kubernetes.io/tls
tls.crt     2.1k    2024-06-10 10:00
tls.key     1.7k    2024-06-10 10:00
```

HOW TO FIX

- Use a valid, non-expired certificate
- Ensure correct TLS secret is referenced
- Match hostname with certificate SAN
- Check cert-manager status and events
- Verify ingress controller TLS setup
- Test HTTPS using openssl or curl
- Configure proper redirects (HTTP → HTTPS)

JUNIOR ENGINEER TIP
Always test from inside and outside the cluster. Use curl, openssl and browser developer tools.

COMMON MISTAKE
It works inside the cluster, so it must be fine. Always test the full external path (DNS → Ingress → Service → Pod) with the correct domain and protocol.

PRODUCTION AWARENESS
TLS and ingress issues can cause downtime, security risks and loss of user trust. Monitor certificate expiry, ingress health and external access regularly.

CONFIGMAPS AND CONFIGURATION

REAL SCENARIOS | REAL TROUBLESHOOTING | REAL INTERVIEW QUESTIONS

ConfigMaps store non-sensitive configuration data for your applications. Issues with ConfigMaps can cause applications to use outdated settings or fail to start. These scenarios test your ability to troubleshoot Kubernetes configuration and understand how changes are applied to running workloads.

TROUBLESHOOT LIKE A REAL ENGINEER

24 APPLICATION RECEIVES OUTDATED CONFIGURATION

SCENARIO

You update a ConfigMap, but the application continues to use the old configuration.

COMMON CAUSES

- Application does not reload config without restart
- ConfigMap is mounted as environment variable (not updated automatically)
- Volume mount does not reflect changes immediately
- Application caches configuration
- Rollout was not triggered after ConfigMap update

KEY COMMANDS

```
kubectl get configmap <name> -o yaml
kubectl describe pod <pod-name>
kubectl get pods -o wide
kubectl rollout restart deployment <name>
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- cat /app/config
```

WHAT TO LOOK FOR

- Check if the ConfigMap was updated
- Verify how the ConfigMap is consumed (env var vs mounted file)
- Confirm if the application supports hot reload
- Check Pod restart time and events
- Validate the configuration inside the container
- Trigger a rollout restart if necessary

EXAMPLE OUTPUT

```
$ kubectl get configmap app-config -o yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: prod
  LOG_LEVEL: info
```

HOW TO FIX

- Update the ConfigMap with correct values
- Use a rollout restart to apply changes
- Use a file mount instead of environment variables if dynamic updates are needed
- Configure the application to reload configuration
- Verify the new configuration inside the Pod
- Monitor logs to confirm the change

25 INCORRECT CONFIGMAP CAUSES APPLICATION STARTUP FAILURE

SCENARIO

You create or update a ConfigMap with incorrect values and the application fails to start.

COMMON CAUSES

- Missing required configuration keys
- Syntax errors in configuration values
- Incorrect file format (YAML, JSON, properties)
- Wrong key names or data structure
- Application expects a specific config format
- ConfigMap mounted to the wrong path
- Environment variables from ConfigMap are invalid or empty

KEY COMMANDS

```
kubectl get configmap <name> -o yaml
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- ls /app/config
kubectl exec -it <pod-name> -- cat /app/config/<file>
kubectl get events --sort-by=.lastTimestamp
```

WHAT TO LOOK FOR

- Error messages in application logs
- Missing or invalid configuration keys
- Incorrect file format or syntax
- Check the actual ConfigMap data
- Verify the mount path and file permissions
- Look for events related to the Pod startup
- Confirm the application's expected configuration format

EXAMPLE OUTPUT

```
$ kubectl logs myapp
Error: could not load configuration
Caused by: missing required key 'DB_HOST'

$ kubectl get configmap app-config -o yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: info   # DB_HOST is missing
```

HOW TO FIX

- Add the missing or correct configuration keys
- Fix syntax or formatting errors
- Ensure the configuration matches the application's expected format
- Redeploy or restart the application
- Validate the configuration inside the container
- Monitor logs to confirm successful startup

JUNIOR ENGINEER TIP
Always validate ConfigMap changes in a non-production environment first.

COMMON MISTAKE
Updating a ConfigMap and expecting the application to automatically reload without a restart.

PRODUCTION AWARENESS
Configuration errors can cause application outages. Use version control, change reviews, and monitoring to prevent and quickly recover from configuration issues.

SECRETS AND CREDENTIALS

REAL SCENARIOS | REAL TROUBLESHOOTING | REAL INTERVIEW QUESTIONS

Kubernetes Secrets store sensitive information such as passwords, API keys, and tokens. Misconfigurations, missing Secrets, or credential rotation can cause Pods to fail or applications to stop working. These scenarios test your ability to troubleshoot Secrets and manage credentials securely in real environments.

TROUBLESHOOT LIKE A REAL ENGINEER

26 POD CANNOT ACCESS REQUIRED SECRET

SCENARIO

You deploy a Pod that requires a Secret, but the Pod cannot access the Secret and fails to start or runs with missing credentials.

COMMON CAUSES

- Secret does not exist or wrong name
- Secret is in a different namespace
- Incorrect key name in the Secret
- Missing or incorrect RBAC permissions
- Secret not mounted correctly (volume or env env var)
- Typo in the manifest
- Application expects a different format

KEY COMMANDS

```
kubectl get secrets
kubectl get secret <name> -o yaml
kubectl describe pod <pod-name>
kubectl describe secret <name>
kubectl get pod <pod-name> -o yaml
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- env | grep <KEY>
```

WHAT TO LOOK FOR

- Secret exists in the correct namespace
- Correct key names and values
- Pod events and error messages
- Volume mounts or env variables are configured
- RBAC permissions if using external secret stores
- Application logs for missing credential errors
- Correct file path and permissions inside the Pod

EXAMPLE OUTPUT

```
$ kubectl describe pod myapp
Events:
  Type     Reason     Message
  ----     ------     -------
  Warning  Failed     Error: secret "db-secret"
                      not found
```

HOW TO FIX

- Create the Secret with the correct name and keys
- Ensure it is in the right namespace
- Update the Pod spec to reference the correct Secret
- Verify volume mounts or env variables
- Check RBAC permissions if using external secrets
- Redeploy the Pod and confirm the application can access the credentials

27 ROTATED CREDENTIALS BREAK AN APPLICATION

SCENARIO

You rotate credentials in a Secret (e.g. database password, API key), but the application stops working after the rotation.

COMMON CAUSES

- Application does not reload credentials
- Secret is updated but Pod still uses old values
- No automatic rollout after Secret change
- Cached or long-lived connections
- Missing restart or configuration reload
- Application expects a different credential format
- External system not updated with new credentials

KEY COMMANDS

```
kubectl get secret <name> -o yaml
kubectl get pods
kubectl describe pod <pod-name>
kubectl rollout restart deployment <name>
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- env | grep <KEY>
```

WHAT TO LOOK FOR

- Secret was updated with new values
- Pod is still running with old environment variables
- Application logs showing authentication errors
- No automatic rollout after Secret change
- Look for cached connections (e.g. database)
- Events showing configuration not reloaded
- Mismatch between new credentials and external system

EXAMPLE OUTPUT

```
$ kubectl get secret db-secret -o yaml
data:
  password: c2VjcmV0MQ==   # old value

$ kubectl describe pod myapp
Events:
  Warning  Failed  Error: authentication failed
                   using provided credentials
```

HOW TO FIX

- Update the Secret with new crodentials
- Trigger a rollout restart of the Deployment
- Ensure the application supports dynamic reload or restart is required
- Update external systems with the new credentials
- Verify the application is using the new values
- Monitor logs and test connectivity after rotation

JUNIOR ENGINEER TIP
Use a rollout restart to apply Secret changes and always test credential rotation in a non-production environment first.

COMMON MISTAKE
Updating a Secret and expecting the application to automatically pick up the new credentials without a restart.

PRODUCTION AWARENESS
Plan credential rotation with minimal downtime, use external secret management tools (e.g. AWS Secrets Manager, HashiCorp Vault), and monitor application health during and after rotation.

PERSISTENT STORAGE

REAL SCENARIOS | REAL TROUBLESHOOTING | REAL INTERVIEW QUESTIONS

Persistent Storage (PV and PVC) allows Pods to store data beyond the Pod lifecycle. Storage issues are common and can prevent Pods from starting, cause data loss, or affect application performance. These scenarios test your ability to troubleshoot Kubernetes storage in real environments.

TROUBLESHOOT LIKE A REAL ENGINEER

28 PVC REMAINS PENDING

SCENARIO

You create a PersistentVolumeClaim (PVC), but it remains in the Pending state and does not bind to a PersistentVolume (PV).

COMMON CAUSES

- No matching PersistentVolume available
- StorageClass does not exist or is incorrect
- Insufficient storage capacity
- Access mode mismatch (RWO, RWX, etc.)
- Volume node affinity or zone constraints
- Dynamic provisioning not working
- Quota or limit restrictions

KEY COMMANDS

```
kubectl get pvc
kubectl describe pvc <pvc-name>
kubectl get pv
kubectl gescribe pv <pv-name>
kubectl get storageclass
kubectl describe storageclass <name>
kubectl get events -n <namespace>
kubectl logs -n kube-system -l app=csi-provisioner
```

WHAT TO LOOK FOR

- PVC events and error messages
- Matching storage class and access modes
- Available PersistentVolumes
- StorageClass provisioner status
- Node affinity and zone constraints
- CSI provisioner logs
- Resource quotas or limits

EXAMPLE OUTPUT

```
$ kubectl get pvc
NAME       STATUS    VOLUME   CAPACITY  ACCESS MODES
data-pvc   Pending   -        -         -

$ kubectl describe pvc data-pvc
Events:
  Type     Reason         Message
  ----     ------         -------
  Warning  Provisioning   no persistent volumes available
                          for this claim and no storage class
                          is set
```

HOW TO FIX

- Check and use the correct StorageClass
- Ensure a matching PV exists or enable dynamic provisioning
- Verify access modes and storage capacity
- Check quota and node constraints
- Review CSI controller logs

29 POD CANNOT MOUNT ITS VOLUME

SCENARIO

The Pod is created, but it fails to start because it cannot mount the volume from the PersistentVolumeClaim.

COMMON CAUSES

- Incorrect volume or mount path
- Permissions or security context issues
- Volume is already in use (ReadWriteOnce)
- CSI driver or node plugin failure
- Wrong file system type
- Missing or incorrect PVC reference
- Node issues (disk, kubelet, mounts)

KEY COMMANDS

```
kubectl describe pod <pod-name>
kubectl describe pvc <pvc-name>
kubectl describe pv <pv-name>
kubectl logs -n kube-system -l app=csi-node
kubectl get events -n <namespace>
kubectl get nodes -o wide
```

WHAT TO LOOK FOR

- MountVolume or FailedMount errors
- PVC and PV status
- Node plugin or CSI driver errors
- Permission denied messages
- File system or device not found errors
- Node health and disk availability

EXAMPLE OUTPUT

```
Events:
  Type     Reason         Message
  ----     ------         -------
  Warning  FailedMount    MountVolume.SetUp failed
                          for volume "data":
                          mount failed: permission denied
```

HOW TO FIX

- Verify the correct PVC and volume name
- Check security context and file permissions
- Ensure the volume is not already attached elsewhere
- Fix CSI driver or node issues
- Verify the mount path inside the container
- Restart the Pod after making changes

30 STORAGE BECOMES FULL

SCENARIO

The application is running, but the storage volume becomes full, causing errors, crashes, or inability to write data.

COMMON CAUSES

- Application generating excessive logs or data
- No log rotation or cleanup
- Large files or backups not removed
- Small volume size
- Monitoring and alerts not configured
- Database or cache using more space than expected

KEY COMMANDS

```
kubectl exec -it <pod-name> -- df -h
kubectl exec -it <pod-name> -- du -sh /*
kubectl exec -it <pod-name> -- du -sh /path
kubectl describe pvc <pvc-name>
kubectl get events -n <namespace>
```

WHAT TO LOOK FOR

- Disk usage and available space
- Large files or directories
- Application logs and error messages
- Rapidly growing data or logs
- PVC size and storage class limits
- Write errors such as "no space left on device"

EXAMPLE OUTPUT

```
$ kubectl exec -it myapp -- df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        20G   20G    0  100% /data

$ kubectl logs myapp
Error: no space left on device
```

HOW TO FIX

- Remove unnecessary files or old logs
- Implement log rotation and cleanup
- Increase PVC size if supported (or create a larger one)
- Monitor disk usage and set up alerts
- Optimize application data storage
- Restart the application if needed

JUNIOR ENGINEER TIP
Always monitor disk usage and set up alerts before storage becomes full.

COMMON MISTAKE
Ignoring disk usage until the application crashes.

PRODUCTION AWARENESS
Storage issues can cause application downtime and data loss. Monitor usage, set alerts, and have a strategy for scaling storage.

HEALTH PROBES

LIVENESS | READINESS | STARTUP

HEALTHY APPLICATIONS HAPPY USERS

Health probes help Kubernetes know when a container is healthy, ready to receive traffic, and when it should be restarted. Misconfigured probes can cause restarts, no traffic, or slow startups.

31 LIVENESS PROBE CAUSES CONTINUOUS RESTARTS

SCENARIO

A Pod keeps restarting. The liveness probe fails and Kubernetes restarts the container repeatedly.

COMMON CAUSES

- Incorrect probe path or port
- Application takes longer to start
- Probe checks the wrong endpoint
- Application temporarily slow
- Aggressive failureThreshold or periodSeconds

KEY COMMANDS

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
kubectl get events
```

WHAT TO LOOK FOR

- Liveness probe failed events
- Restart count increasing
- Error messages in application logs
- Probe configuration in the manifest
- Application startup time

EXAMPLE OUTPUT (describe pod)

```
Liveness:   http-get http://:8080/health
  Initial delay seconds:  10
  Period seconds:         10
  Timeout seconds:        1
Events:
 Warning Unhealthy  Liveness probe failed:
 HTTP probe failed with status code: 500
```

HOW TO FIX

- Verify the probe path and port
- Increase initialDelaySeconds
- Adjust timeoutSeconds or failureThreshold
- Ensure the application is actually healthy
- Test the endpoint from inside the container

32 READINESS PROBE REMOVES HEALTHY-LOOKING PODS FROM TRAFFIC

SCENARIO

Pods are running, but they are not receiving traffic because the readiness probe keeps failing.

COMMON CAUSES

- Readiness endpoint returns non-200
- Application not fully initialized
- Dependency not ready (database, cache)
- Probe path or port misconfigured
- Application takes time to become ready

KEY COMMANDS

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl get endpoints <service-name>
kubectl logs <pod-name>
kubectl exec -it <pod-name> -- curl localhost:8080/health
```

WHAT TO LOOK FOR

- Readiness probe failed events
- Service shows no or few endpoints
- Endpoint returns error or timeout
- Application logs showing dependency errors
- Probe configuration

EXAMPLE OUTPUT (get endpoints)

NAME	ENDPOINTS	AGE
myapp	<none>	10m

HOW TO FIX

- Verify the readiness endpoint
- Ensure application dependencies are ready
- Check service and endpoint configuration
- Increase initialDelaySeconds if needed
- Confirm the application returns 200 OK

33 SLOW-STARTING APPLICATION NEEDS A STARTUP PROBE

SCENARIO

An application takes a long time to start. Liveness probe kills it before it is fully initialized.

COMMON CAUSES

- Application startup time is long
- Liveness probe starts too early
- No startup probe configured
- Heavy initialization (large files, migrations)
- Readiness probe also failing early

KEY COMMANDS

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get events
```

WHAT TO LOOK FOR

- Container killed before startup completes
- Liveness probe failures during initialization
- Application logs showing startup process
- Probe timings in the manifest

EXAMPLE (startup probe config)

```
startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
```

HOW TO FIX

- Add a startup probe
- Set appropriate failureThreshold and periodSeconds
- Keep liveness probe for runtime checks
- Ensure readiness probe is configured correctly

JUNIOR ENGINEER TIP
Use startupProbe for slow-starting apps. It gives the application time to initialize before liveness checks begin.

PRODUCTION AWARENESS
Misconfigured probes can cause outages or no traffic to your application. Always test probe endpoints in real environments before deploying.

COMMON MISTAKE
Using liveness probe to check if the app is ready. Liveness checks if the app is alive. Readiness checks if it can receive traffic.

REMEMBER
Liveness = Restart
Readiness = Traffic
Startup = Give Time

NODES AND KUBELET

KEEP THE CLUSTER HEALTHY, STABLE AND AVAILABLE

HEALTHY NODES = HEALTHY APPLICATIONS

Nodes run your workloads. The kubelet runs on each node and reports the node and Pod status to the control plane. When nodes or kubelet have issues, workloads can stop, restart or be evicted.

34 NODE BECOMES NOTREADY

SCENARIO

A node shows NotReady and new Pods are not being scheduled to it.

COMMON CAUSES

- Network connectivity issues
- Kubelet stopped or unhealthy
- Node out of disk, memory or inode resources
- Container runtime (containerd/Docker) issues
- TLS certificate or kubelet authentication issues

KEY COMMANDS

```
kubectl get nodes
kubectl describe node <node-name>
kubectl get events --field-selector
 involvedObject.kind=Node
kubectl logs -n kube-system kubelet
```

WHAT TO LOOK FOR

- Node conditions (Ready, MemoryPressure)
- Events and error messages
- Kubelet and runtime logs
- Network and DNS issues
- Resource exhaustion on the node

EXAMPLE OUTPUT

NAME	STATUS	ROLES	AGE
ip-10-0-1-123	NotReady	<none>	2d

HOW TO FIX

- Check kubelet and container runtime
- Verify node network and DNS
- Check disk, memory and inode usage
- Review kubelet configuration and certificates
- Fix underlying infrastructure issue
- Restart kubelet (if safe and appropriate)

35 KUBELET STOPS REPORTING CORRECTLY

SCENARIO

The kubelet is not reporting node or Pod status correctly. Node details are missing or outdated.

COMMON CAUSES

- Kubelet service stopped or crashed
- Kubelet cannot reach the API server
- TLS certificate expired or invalid
- High resource usage on the node
- Container runtime not responding

KEY COMMANDS

```
systemctl status kubelet
journalctl -u kubelet -n 50
kubectl get nodes
kubectl describe node <node-name>
kubectl get pods -A -o wide
```

WHAT TO LOOK FOR

- Kubelet log errors
- Certificate or authentication errors
- API server connectivity issues
- Runtime errors (containerd/Docker)
- Resource pressure or node instability

EXAMPLE OUTPUT (journalctl)

```
Unable to register node with API server:
x509: certificate has expired or is not yet
valid
```

HOW TO FIX

- Restart kubelet service
- Renew or fix certificates
- Check API server connectivity
- Verify kubelet configuration
- Fix container runtime issues
- Ensure the node has enough resources

36 WORKLOADS ARE EVICTED BECAUSE OF NODE PRESSURE

SCENARIO

Pods are being evicted from the node. Events show node pressure due to high resource usage.

COMMON CAUSES

- Disk pressure (low disk space or inodes)
- Memory pressure (high memory usage)
- PID pressure (too many processes)
- Application using more resources than limits
- Large logs or temporary files

KEY COMMANDS

```
kubectl describe node <node-name>
kubectl get events --sort-by=.lastTimestamp
kubectl top node
kubectl top pods -A --sort-by=memory
df -h
free -h
```

WHAT TO LOOK FOR

- Node conditions (DiskPressure, MemoryPressure)
- Eviction events and messages
- High resource usage on node or Pods
- Large log files or emptyDir usage
- Pods without proper resource limits

EXAMPLE OUTPUT (events)

Reason	Message	Age
Evicted	The node was low on memory and required eviction.	2m
Evicted	The node was low on disk space.	5m

HOW TO FIX

- Free up disk space or increase node storage
- Reduce memory usage or increase node memory
- Set appropriate requests and limits
- Clean up logs and temporary files
- Use monitoring and alerts to prevent recurrence

JUNIOR ENGINEER TIP
Always check events first. They usually tell you exactly why a node is NotReady or why a Pod was evicted.

PRODUCTION AWARENESS
Monitor node resources (CPU, memory, disk, inodes) and kubelet health. Set alerts before users are affected.

COMMON MISTAKE
Ignoring node events and focusing only on Pod status. Node issues can affect multiple workloads.

REMEMBER
No healthy nodes = No running applications.

RBAC AND AUTHORIZATION

CONTROL WHO CAN DO WHAT IN KUBERNETES

LEAST PRIVILEGE = MORE SECURE CLUSTERS

Kubernetes uses Role-Based Access Control (RBAC) to authorize users, ServiceAccounts and applications. Proper RBAC ensures only the right people and workloads have the right permissions.

37 USER RECEIVES FORBIDDEN ERROR

SCENARIO

A user runs a kubectl command and gets a Forbidden error. They cannot perform the action even though they are authenticated.

COMMON CAUSES

- User does not have the required Role or ClusterRole
- RoleBinding or ClusterRoleBinding is missing or incorrect
- Wrong namespace in the binding
- User is bound to the wrong group
- Trying to access a cluster-scoped resource without cluster permissions

KEY COMMANDS

```
kubectl auth can-i <verb> <resource>
kubectl auth can-i <verb> <resource> -n <ns>
kubectl get role,clusterrole
kubectl get rolebinding,clusterrolebinding
kubectl describe rolebinding <name>
kubectl describe clusterrolebinding <name>
kubectl config view --minify
```

WHAT TO LOOK FOR

- Exact error message and resource
- Which user or group is being used
- Role or ClusterRole rules
- RoleBinding or ClusterRoleBinding
- Namespace scope vs cluster scope
- If the user is in the correct group

EXAMPLE OUTPUT

```
Error from server (Forbidden):
users "john" cannot list pods
in the namespace "dev"
```

HOW TO FIX

- Grant the required permissions using a Role or ClusterRole
- Create or update the RoleBinding or ClusterRoleBinding
- Ensure the user or group is correct
- Use the least privilege permissions
- Verify with kubectl auth can-i

JUNIOR ENGINEER TIP
Authentication means who you are. Authorization means what you are allowed to do. You can be logged in and still not have permission.

COMMON MISTAKE
Giving cluster-admin to fix a permission issue without understanding the actual required permissions.

38 SERVICEACCOUNT LACKS REQUIRED PERMISSIONS

SCENARIO

A Pod or application using a ServiceAccount fails with a Forbidden error when trying to access the Kubernetes API or other resources.

COMMON CAUSES

- ServiceAccount has no Role or ClusterRole bound
- Role rules do not include the required resource or verb
- RoleBinding is in the wrong namespace
- Using the default ServiceAccount without permissions
- Application is trying to access cluster-wide resources

KEY COMMANDS

```
kubectl get serviceaccount -n <ns>
kubectl describe serviceaccount <name> -n <ns>
kubectl get role,rolebinding -n <ns>
kubectl auth can-i <verb> <resource> \
      --as=system:serviceaccount:<ns>:<name>
kubectl logs <pod-name> -n <ns>
kubectl describe pod <pod-name> -n <ns>
```

WHAT TO LOOK FOR

- ServiceAccount name used by the Pod
- Role or ClusterRole rules
- RoleBinding or ClusterRoleBinding
- Namespace scope
- API error messages in application logs
- Whether the permissions match the required resource and verb

EXAMPLE OUTPUT

```
Error from server (Forbidden):
serviceaccounts "app-sa" cannot get
configmaps in the namespace "prod"
```

HOW TO FIX

- Create a Role or ClusterRole with the required permissions
- Bind it to the ServiceAccount using a RoleBinding or ClusterRoleBinding
- Ensure the binding is in the correct namespace
- Verify permissions with kubectl auth can-i
- Test the application again

PRODUCTION AWARENESS
Always follow the principle of least privilege. Only grant the permissions the application needs.

NETWORK POLICIES

CONTROL TRAFFIC BETWEEN PODS

SECURE BY DESIGN = STRONGER CLUSTERS

Network Policies allow you to define which Pods can communicate with each other. They help improve security, reduce the blast radius, and enforce the principle of least privilege.

39 APPLICATION LOSES CONNECTIVITY AFTER NETWORKPOLICY DEPLOYMENT

SCENARIO

An application was working, but after deploying a NetworkPolicy, it loses connectivity to another service or external resource.

COMMON CAUSES

- NetworkPolicy is too restrictive
- Missing ingress or egress rule
- Incorrect labels or namespace selector
- Traffic to DNS is blocked
- Egress to external services not allowed
- Default deny policy applied without required exceptions

KEY COMMANDS

```
kubectl get networkpolicy -A
kubectl describe networkpolicy <name>
kubectl get pods --show-labels
kubectl get svc -n <ns>
kubectl exec -it <pod> -- nslookup <svc>
kubectl exec -it <pod> -- curl <svc>:<port>
kubectl get events -n <ns>
```

WHAT TO LOOK FOR

- Which policies apply to the Pod
- Pod and namespace labels
- Allowed ingress and egress rules
- DNS resolution (CoreDNS)
- Events or connection timeout errors
- Whether traffic is allowed by selectors
- Any default deny policies

EXAMPLE OUTPUT

```
$ kubectl exec -it pod -- curl backend:8080
  curl: (7) Failed to connect to backend
  port 8080: Connection timed out

$ kubectl get networkpolicy
  NAME          POD-SELECTOR     AGE
  default-deny  app=frontend     2m
```

HOW TO FIX

- Review and update NetworkPolicy rules
- Allow required ingress/egress traffic
- Ensure DNS (UDP/TCP 53) is allowed
- Verify labels match the intended Pods
- Add egress rules for external services if needed
- Test connectivity after changes

JUNIOR ENGINEER TIP
When using a default deny policy, always allow DNS traffic first. Without DNS, many applications will not work.

COMMON MISTAKE
Applying a default deny NetworkPolicy without fully understanding the required traffic flows.

40 FRONTEND CANNOT COMMUNICATE WITH BACKEND

SCENARIO

The frontend application cannot reach the backend service. Both Pods are running, but the connection fails after a NetworkPolicy was applied.

COMMON CAUSES

- Missing ingress rule on backend
- Incorrect podSelector or namespaceSelector
- Wrong port in the rule
- Labels do not match the Pods
- Egress rule missing on frontend
- NetworkPolicy applied to the wrong namespace

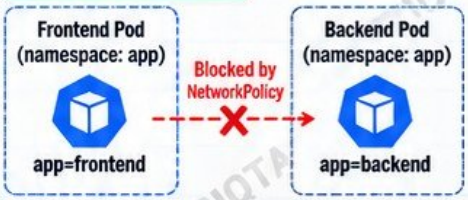
KEY COMMANDS

```
kubectl get pods --show-labels -n <ns>
kubectl get networkpolicy -n <ns>
kubectl describe networkpolicy <name>
kubectl exec -it <frontend-pod> -- curl backend:8888
kubectl exec -it <frontend-pod> -- telnet backend 8888
kubectl get svc -n <ns>
kubectl get endpoints <svc> -n <ns>
```

WHAT TO LOOK FOR

- Labels on frontend and backend Pods
- Ingress rules on backend
- Egress rules on frontend (if restricted)
- Service name, port and endpoints
- Connection timeout or "No route to host"
- Policy applied to the correct namespace

EXAMPLE DIAGRAM



HOW TO FIX

- Add an ingress rule on the backend to allow traffic from the frontend (using podSelector or namespaceSelector)
- Ensure the correct port is allowed
- Verify labels match the actual Pods
- Check both ingress (backend) and egress (frontend)
- Test connectivity again after applying the changes

PRODUCTION AWARENESS
NetworkPolicies are additive. Start with allow rules for required traffic and test in a staging environment before applying in production.

REMEMBER
If DNS is blocked, service names will not resolve. Always allow DNS (UDP/TCP 53) in your NetworkPolicy.

REAL-WORLD IMPACT
Incorrect NetworkPolicies can silently break communication between services, causing application errors even though all Pods are running.

STATEFUL WORKLOADS

PERSISTENT IDENTITY | PERSISTENT STORAGE | RELIABLE APPLICATIONS

STATE MATTERS IN PRODUCTION

Stateful workloads such as databases, message queues and distributed systems require stable identities, persistent storage and ordered operations. Understanding StatefulSets, PVCs and storage is critical for running reliable stateful applications in Kubernetes.

41 STATEFULSET POD CANNOT RECOVER CORRECTLY

SCENARIO

A StatefulSet Pod is stuck and cannot recover correctly after being deleted or restarted. It may stay in Pending or CrashLoopBackOff.

COMMON CAUSES

- PVC is not bound or missing
- Storage issue (disk, permissions)
- Pod identity or ordinal problems
- Readiness or startup probe failures
- Node or storage class issues
- Application data corruption
- Incorrect volumeMount or permissions

KEY COMMANDS

```
kubectl get statefulset
kubectl get pods -l app=<name>
kubectl describe pod <pod-name>
kubectl get pvc
kubectl get pv
kubectl logs <pod-name> --previous
kubectl get events --sort-by='.lastTimestamp'
```

WHAT TO LOOK FOR

- Pod events and error messages
- PVC status (Bound/Pending)
- Storage class and PV information
- Volume mount errors in describe output
- Application logs and startup errors
- Node and storage health
- Whether the correct ordinal is used

EXAMPLE OUTPUT (describe pod)

```
Events:
 Warning  FailedMount   2m
   Unable to attach or mount volumes:
   persistentvolumeclaim "data-db-0"
   not found
```

HOW TO FIX

- Ensure the PVC exists and is Bound
- Check the storage class and PV
- Verify volumeMount paths and permissions
- Fix application configuration or data issues
- Check node and storage health
- If needed, delete and recreate the Pod (not the PVC)
- Confirm the Pod starts with the correct ordinal

JUNIOR ENGINEER TIP
StatefulSet Pods have stable network identities and PVCs. Do not delete PVCs unless you intentionally want to lose the data.

COMMON MISTAKE
Deleting the PVC to fix a stuck Pod. This will permanently remove the application data.

42 STATEFUL APPLICATION LOSES EXPECTED STORAGE OR IDENTITY BEHAVIOR

SCENARIO

A stateful application loses its data or identity. After a restart or rescheduling, the Pod comes up but the application has lost data, configuration or its expected node identity.

COMMON CAUSES

- Using ephemeral storage instead of PVC
- PVC was accidentally deleted
- Wrong storage class or reclaim policy
- Application writes to the wrong path
- Multiple Pods sharing the same PVC
- Misconfigured StatefulSet or Deployment
- Data not persisted because of container config

KEY COMMANDS

```
kubectl get pvc
kubectl describe pvc <pvc-name>
kubectl get pv
kubectl describe pv  <pv-name>
kubectl get statefulset
kubectl describe statefulset <name>
kubectl exec -it <pod-name> -- df -h
kubectl exec -it <pod-name> -- ls -l <path>
```

WHAT TO LOOK FOR

- Whether the Pod is using a PVC
- Storage class and reclaim policy
- Mounted path inside the container
- Application data directory and file ownership
- Whether the correct ordinal and Pod name is used
- Signs of data being written to an ephemeral path
- Any recent changes to the StatefulSet or storage

EXAMPLE OUTPUT (PVC)

NAME	STATUS	VOLUME	CAPACITY
data-db-0	Bound	pv-abc123	50Gi

HOW TO FIX

- Use a PersistentVolumeClaim for stateful data
- Verify the correct storage class and reclaim policy
- Ensure the application writes to the mounted volume
- Check file permissions and ownership
- Use StatefulSet (not Deployment) for stateful apps
- Recover data from backups if necessary
- Validate that the Pod identity and ordinal are correct

PRODUCTION AWARENESS
Stateful data is critical. Always use PVCs, monitor storage usage and have backups.

REMEMBER
A Pod can be replaced. Your data should not be. Use persistent storage and test recovery procedures regularly.

REAL-WORLD IMPACT
Losing state can cause downtime, data loss and manual recovery. Design for persistence from the start.

JOBS AND CRONJOBS

RUN TASKS ONCE OR ON A SCHEDULE

AUTOMATE RELIABLE TASKS = MORE STABLE SYSTEMS

Jobs run Pods to completion. CronJobs run Jobs on a schedule. They are commonly used for batch processing, backups, reports and data pipelines. Understanding how to troubleshoot them is essential for reliable operations in Kuberentes.

43 JOB REPEATEDLY FAILS

SCENARIO

A Job keeps failing and does not complete successfully. The Pods restart or exit with errors.

COMMON CAUSES

- Application error or incorrect command
- Missing ConfigMap or Secret
- Incorrect environment variables
- Image or dependency issues
- Insufficient resources (CPU or memory)
- Wrong restartPolicy or backoffLimit
- External service or database not reachable
- Permission issues (RBAC)
- Network or DNS problems

KEY COMMANDS

```
kubectl get jobs
kubectl describe job <job-name>
kubectl get pods --selector=job-name=<job-name>
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
kubectl get events --sort-by=.lastTimestamp
```

WHAT TO LOOK FOR

- Job status and completion count
- Pod exit code and error messages
- Events (Failed, BackoffLimitExceeded)
- Container logs
- Missing configuration or secrets
- Resource limits and node conditions
- Application specific errors

EXAMPLE OUTPUT (describe job)

```
Name:           data-processing
Namespece:      batch
Completions:    1
Failed:         3
Backoff Limit:  6
Events:
  Warning  BackoffLimitExceeded  2m
  Job has reached the specified backoff limit
```

HOW TO FIX

- Check Pod logs and fix the application error
- Verify required ConfigMaps, Secrets and env vars
- Increase resources if needed
- Fix image or dependency issues
- Check network connectivity to external services
- Adjust restartPolicy or backoffLimit
- Test the command locally if possible

JUNIOR ENGINEER TIP
A Job is not a long running service. Check the logs first. Most Job failures are caused by application errors or missing configuration.

COMMON MISTAKE
Forgetting to check the Pod logs. The Job will keep retrying and eventually hit BackoffLimit.

44 CRONJOB DOES NOT EXECUTE WHEN EXPECTED

SCENARIO

A CronJob does not run at the expected time. No new Job or Pod is created.

COMMON CAUSES

- Incorrect cron schedule format
- Time zone confusion (uses controller time)
- Suspended CronJob (suspend: true)
- Missed schedule due to controller downtime
- Concurrency policy blocking new runs
- Starting deadline missed
- RBAC or permission issues
- Image pull or configuration errors
- CronJob disabled or deleted

KEY COMMANDS

```
kubectl get cronjobs
kubectl describe cronjob <name>
kubectl get jobs --from=cronjob/<name>
kubectl get events --field-selector
 involvedObject.kind=CronJob
kubectl logs <pod-name>
kubectl get cm kube-controller-manager -n kube-system
o yaml | grep -i timezone
```

WHAT TO LOOK FOR

- Schedule expression and time zone
- Last scheduled time and last successful time
- Events and error messages
- Suspend field value
- Concurrency policy (Allow, Forbid, Replace)
- Starting deadline seconds
- Whether a Job was created but failed

EXAMPLE OUTPUT (describe cronjob)

```
Name:                   backup-job
Schedule:               0 2 * * *
Suspend:                False
Concurrency Policy:     Forbid
Last Schedule Time:     <none>
Events:
  Normal   Scheduled    8h  Successfully scheduled job
  Warning  MissSchedule 8h  Missed scheduled time
```

HOW TO FIX

- Verify the cron schedule syntax
- Check the correct time zone (controller time)
- Ensure suspend is set to false
- Review concurrency policy and starting deadline
- Check controller manager logs if needed
- Verify the CronJob has permission to create Jobs
- Confirm that the created Job runs successfully

PRODUCTION AWARENESS
CronJobs are used for critical tasks like backups and financial reports. Always monitor execution and set up alerting for missed runs.

REMEMBER
Kubernetes uses the controller manager time zone (not your local time). A valid schedule expression does not guarantee a run if the controller is down.

REAL-WORLD IMPACT
Missed CronJobs can lead to missed backups, incomplete reports and data loss. Always monitor and alert on failed or missed runs.

SCALING AND AVAILABILITY

SCALE SMARTER. KEEP APPLICATIONS AVAILABLE

SCALE FOR DEMAND = HIGHER AVAILABILITY

Kubernetes provides multiple scaling options such as Horizontal Pod Autoscaler (HPA), manual scaling and cluster autoscaling. Proper scaling ensures your applications can handle traffic while remaining highly available.

45 HPA DOES NOT SCALE THE APPLICATION

SCENARIO

You configured an HPA but the number of Pods does not increase even when CPU or memory usage is high.

COMMON CAUSES

- Metrics Server not installed or not working
- Incorrect metric type or threshold
- Wrong resource name in HPA (cpu vs memory)
- Application is not generating enough metrics
- HPA is targeting the wrong Deployment
- minReplicas equals maxReplicas
- Custom metrics not available
- Namespace or RBAC issues

KEY COMMANDS

```
kubectl get hpa
kubectl describe hpa <hpa-name>
kubectl get deployment <name>
kubectl top pods
kubectl get pods -l app=<name>
kubectl get --raw /apis/metrics.k8s.io/v1beta1
kubectl logs -n kube-system deploy/metrics-server
```

WHAT TO LOOK FOR

- HPA status and current metrics
- Events and error messages
- Metrics Server is running and collecting metrics
- Correct target CPU/memory utilization
- Current vs desired replicas
- minReplicas and maxReplicas values

EXAMPLE OUTPUT (kubectl get hpa)

NAME	REFERENCE	TARGETS	MIN	MAX	REPLICAS	AGE
app-hpa	Deployment/app	5%/80%	2	10	2	10m

```
kubectl describe hpa app-hpa
Conditions:
```

Type	Status	Reason	Message
AbleToScale	True	Success	the HPA is able to scale
ScalingActive	False	FailedGetMetrics	unable to get metrics from resource metrics API

HOW TO FIX

- Install or fix Metrics Server
- Verify metric type and thresholds
- Check application is generating metrics
- Ensure correct Deployment and namespace
- Adjust minReplicas / maxReplicas
- Check events and controller logs
- Use custom/external metrics if required
- Test by generating load

JUNIOR ENGINEER TIP
The HPA only scales based on metrics it can see. If metrics are missing or incorrect, it will not scale even if your application is under heavy load.

COMMON MISTAKE
Setting the HPA and expecting it to work without verifying that metrics are available and correct.

46 SCALING CREATES PODS THAT REMAIN PENDING

SCENARIO

You scale a Deployment or HPA increases replicas, but the new Pods remain in Pending state and do not start.

COMMON CAUSES

- Insufficient cluster resources (CPU, memory)
- Node affinity, taints or tolerations
- Pod anti-affinity rules
- Insufficient quotas or limits
- PVCs not bound (for stateful workloads)
- Image pull issues
- Cluster autoscaler not configured or not working
- Scheduling constraints (topology, zones)

KEY COMMANDS

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl get nodes
kubectl describe nodes
kubectl get events --sort-by=.lastTimestamp
kubectl get pvc
kubectl get storageclass
kubectl get quota -n <namespace>
```

WHAT TO LOOK FOR

- Events and scheduling messages
- Insufficient CPU or memory
- Taints and tolerations
- Node affinity / anti-affinity rules
- PVC status (Pending)
- Image pull errors
- Resource quotas or limits
- Cluster autoscaler status

EXAMPLE OUTPUT (kubectl describe pod)

```
Events:
```

Type	Reason	Age	From	Message
Warning	FailedScheduling	2m	default-scheduler	0/3 nodes are available: 2 Insufficient cpu, 1 node(s) had taint {node-role.kubernetes.io/master: NoSchedule}.

HOW TO FIX

- Add more node resources or enable cluster autoscaler
- Check and adjust resource requests and limits
- Review and update node affinity, taints and tolerations
- Ensure PVCs are bound and storage is available
- Verify image can be pulled (registry access, secrets)
- Check and increase quotas or limits
- Relax scheduling constraints if appropriate
- Confirm nodes are healthy and ready

PRODUCTION AWARENESS
Scaling without available resources will not work. Monitor cluster capacity and set up cluster autoscaling.

REMEMBER
Pending Pods mean a scheduling problem, not an application problem. Always check events for the exact reason.

REAL-WORLD IMPACT
If your application cannot scale, it may not handle increased traffic, leading to downtime and poor user experience.

PRODUCTION INCIDENT TROUBLESHOOTING

PRODUCTION ISSUES MAKE BETTER ENGINEERS

DETECT. INVESTIGATE. RESOLVE. PREVENT.

Production incidents happen. The key is a structured approach to quickly identify the root cause, reduce impact and restore service. These scenarios test your real-world troubleshooting skills.

47 APPLICATION SUDDENLY RETURNS ERRORS AFTER A DEPLOYMENT

SCENARIO

After a successful deployment, the application starts returning errors (e.g. 5xx or increased 4xx) even though the Pods are running.

COMMON CAUSES

- Bug in the new application version
- Incorrect configuration or environment variables
- Database or external service issues
- Incompatible dependencies
- Readiness probe misconfiguration
- Traffic routing issue (Service, Ingress, or mesh)
- Resource limits causing restarts
- Missing secrets or ConfigMaps
- Schema or migration failures

KEY COMMANDS

```
kubectl get pods
kubectl logs <pod-name> -n <ns>
kubectl describe pod <pod-name>
kubectl get events -n <ns> --sort-by=.lastTimestamp
kubectl get deployment <name> -n <ns>
kubectl rollout history deployment <name>
kubectl get svc,ingress -n <ns>
kubectl exec -it <pod-name> -- /bin/sh
```

WHAT TO LOOK FOR

- Application logs and stack traces
- Errors after the new deployment timestamp
- Configuration or environment variable issues
- Database or external API connectivity
- Readiness and liveness probe status
- Service, Ingress or routing changes
- Event messages and warning signs
- Recent changes in ConfigMaps, Secrets or images

EXAMPLE OUTPUT (logs)

```
2024-06-10T10:15:23Z  ERROR
java.lang.Exception: Failed to connect
to database
Caused by: org.postgresql.util.PSQLException:
Connection refused
```

HOW TO FIX

- Check application logs and identify the error
- Compare with previous working version
- Validate configuration, secrets and environment variables
- Check database and external service connectivity
- Rollback if necessary: kubectl rollout undo deployment <name>
- Fix the issue, redeploy and monitor
- Verify the application is healthy and serving traffic

JUNIOR ENGINEER TIP
Always check what changed. Look at recent deployments, ConfigMaps, Secrets, environment variables and database migrations.

COMMON MISTAKE
Assuming the Pods are running means the application is healthy. Pods can be healthy while the app still fails.

48 MULTIPLE PODS FAIL SIMULTANEOUSLY WHILE KUBERNETES STILL REPORTS THE CLUSTER AS RUNNING

SCENARIO

Multiple Pods across one or more Deployments fail at the same time, but the Kubernetes cluster itself still shows as running and healthy.

COMMON CAUSES

- Bad image pushed to multiple workloads
- Shared dependency failure (database, cache, API)
- ConfigMap or Secret change affecting multiple Pods
- Node or network issue (partial outage)
- Resource exhaustion (CPU, memory, disk)
- DNS resolution failure
- External service outage
- Third-party service or cloud provider issue

KEY COMMANDS

```
kubectl get pods -A
kubectl get events -A --sort-by=.lastTimestamp
kubectl get deployments -A
kubectl describe pod <pod-name>
kubectl logs <pod-name> -n <ns>
kubectl get nodes
kubectl top nodes
kubectl get svc,endpoints -A
kubectl get configmap,secret -A
kubectl get ingress -A
```

WHAT TO LOOK FOR

- Common error patterns across multiple Pods
- Recent changes (deployments, ConfigMaps, Secrets)
- Node status and resource usage
- Connectivity to shared dependencies
- DNS or network issues
- Events showing widespread failures
- Cloud provider or external service status

EXAMPLE OUTPUT (multiple pods)

NAME	READY	STATUS	RESTARTS	AGE
app-1	0/1	CrashLoopBackOff	5	2m
app-2	0/1	CrashLoopBackOff	5	2m
app-3	0/1	CrashLoopBackOff	5	2m
worker	0/1	Error	3	2m

HOW TO FIX

- Identify the common root cause across all failing Pods
- Check recent deployments and configuration changes
- Verify the health of external dependencies
- Check node health and cluster events
- Roll back or fix the faulty deployment or configuration
- Ensure sufficient cluster resources
- Communicate the incident and monitor recovery
- Implement additional alerting to detect similar issues early

PRODUCTION AWARENESS
A cluster can be "healthy" while your applications are down. Always monitor both infrastructure and applications.

REMEMBER
Look for patterns. Multiple Pods failing at the same time usually indicates a shared dependency, configuration change or external service issue.

REAL-WORLD IMPACT
Fast and accurate incident troubleshooting reduces downtime, protects users and builds trust in your platform.

MULTI-LAYER TROUBLESHOOTING

FIND THE REAL CAUSE. SOLVE ACROSS THE STACK.

THINK ACROSS LAYERS = SOLVE FASTER

Real production issues are rarely a single problem. They can involve multiple layers such as DNS, networking, workloads, nodes, configurations and external dependencies. This section helps you troubleshoot complex scenarios step by step.

49 USERS REPORT INTERMITTENT APPLICATION FAILURES ACROSS MULTIPLE REPLICAS

SCENARIO

Users report random failures, timeouts or errors when accessing the application. The issue is intermittent and affects multiple replicas, making it difficult to reproduce.

COMMON CAUSES

- Unstable network or DNS resolution
- Intermittent dependency failures (database, external API)
- Resource limits or CPU/memory spikes
- Readiness probe or health check issues
- Load balancer or ingress timeouts
- Application bugs or memory leaks
- Node or underlying infrastructure issues
- Configuration or environment inconsistencies

KEY COMMANDS

```
kubectl get pods -l app=<name>
kubectl describe pod <pod-name>
kubectl logs <pod-name> --previous
kubectl top pods
kubectl get events --sort-by=.lastTimestamp
kubectl get svc,ep -n <ns>
kubectl get ingress
kubectl get endpoints <service>
```

WHAT TO LOOK FOR

- Application logs and error patterns
- Pod restarts or OOMKilled events
- Resource usage (CPU, memory)
- Readiness and liveness probe status
- Network errors and DNS resolution
- External service or database latency
- Load balancer or ingress errors
- Correlation with recent deployments
- Trends (time, affected replicas, frequency)

EXAMPLE OUTPUT (logs)

```
2024-06-10T10:15:23Z  ERROR  timeout
2024-06-10T10:15:25Z  WARN   connection
2024-06-10T10:15:26Z  ERROR  upstream error
2024-06-10T10:15:30Z  INFO   retrying request
```

HOW TO FIX

- Check application and dependency logs
- Verify DNS and network connectivity
- Review resource requests and limits
- Check readiness/liveness probe configuration
- Investigate external service health
- Roll back recent changes if needed
- Tune timeouts, retries and connection pools
- Monitor and set up proper alerts
- Perform load testing to identify bottlenecks

JUNIOR ENGINEER TIP
Look at the whole request flow (ingress → service → pod → application → external dependencies). Intermittent issues are often caused by network, DNS or external services, not just the application.

COMMON MISTAKE
Focusing only on the application Pod logs and ignoring networking, DNS, dependencies or recent changes.

50 PRODUCTION SERVICE IS UNAVAILABLE AND THE CAUSE COULD BE DNS, NETWORKING, WORKLOAD, NODE, CONFIGURATION OR DEPENDENCY FAILURE

SCENARIO

A production service is completely unavailable. Users cannot access the application. The root cause could be any layer including DNS, networking, workload, node, configuration or an external dependency.

COMMON CAUSES

- DNS resolution or record issues
- Network or firewall rules blocking traffic
- Ingress, load balancer or service misconfiguration
- Pods not running or in CrashLoopBackOff
- Node failure or NotReady status
- Configuration or secret issues
- External dependency outage (database, API)
- Certificate or TLS problems
- Cloud provider or infrastructure issues

KEY COMMANDS

```
kubectl get pods -A
kubectl get svc,ep -A
kubectl get ingress -A
kubectl get nodes
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get events -A
kubectl get configmap,secret -A
dig <domain>
nslookup <domain>
curl -v <service-url>
traceroute <service-url>
```

WHAT TO LOOK FOR

- DNS resolution and correct IP
- Network connectivity and firewall rules
- Service, ingress and endpoint status
- Pod and node health
- Recent changes in deployment or configuration
- External dependency health and response time
- TLS/certificate errors
- Cloud provider status or region issues
- Event logs and cluster-wide alerts

TROUBLESHOOTING FLOW

1. Check DNS resolution (dig, nslookup)
2. Verify network connectivity (ping, traceroute, curl)
3. Check load balancer / ingress / service
4. Check pod status and logs
5. Check node health and cluster events
6. Review configuration and secrets
7. Check external dependencies
8. Identify recent changes
9. Implement fix and verify end-to-end
10. Set up monitoring and alerts

HOW TO FIX

- Fix DNS records or routing issues
- Update network rules or security groups
- Correct service, ingress or load balancer configuration
- Restart or redeploy workloads
- Replace or repair unhealthy nodes
- Fix configuration, secrets or certificates
- Restore external dependencies
- Roll back recent changes if necessary
- Validate the service is fully accessible

PRODUCTION AWARENESS
Production outages can have multiple root causes. Always investigate across all layers and communicate clearly with the team.

REMEMBER
A running cluster does not mean your application is healthy. Always verify the full request path from user to dependency.

REAL-WORLD IMPACT
Multi-layer troubleshooting is a critical skill for SRE and production engineers. It reduces downtime and builds trust.